به نام خدا

۱. قاعدهٔ زنجیره‌ای و ماتریس‌های ژاکوبین در شبکه‌های عصبی

فرض کنید $f : \mathbb{R}^n \to \mathbb{R}^m$ و $g : \mathbb{R}^m \to \mathbb{R}^k$ دو تابع مشتق‌پذیر باشند و تابع ترکیبی $h : \mathbb{R}^n \to \mathbb{R}^k$ به‌صورت $h(x) = g(f(x))$ تعریف شده باشد.

(آ) ماتریس ژاکوبین تابع $h(x)$ یعنی $J_h(x)$ را برحسب ماتریس‌های ژاکوبین $J_f(x)$ و $J_g(f(x))$ به‌دست آورید. ابعاد هر یک از این ماتریس‌ها را مشخص کنید.

(ب) فرض کنید توابع f و g به‌صورت زیر تعریف شده باشند:

$$f(x) = \mathrm{ReLU}(Ax + b), \qquad g(z) = Cz$$

که در آن $A \in \mathbb{R}^{m\times n}$، $b \in \mathbb{R}^m$، و $C \in \mathbb{R}^{k\times m}$ ماتریس‌ها و بردار ثابت هستند، و تابع فعال‌ساز ReLU به‌صورت مولفه‌ای روی بردار ورودی $u \in \mathbb{R}^m$ عمل می‌کند:

$$\mathrm{ReLU}(u_i) = \max(0, u_i)$$

i. ضابطهٔ صریح تابع ترکیبی $h(x) = g(f(x))$ را بنویسید.

ii. مشتق تابع برداری $\mathrm{ReLU}(u)$ را با استفاده از یک ماتریس قطری $\mathrm{diag}(\cdot)$ بیان کنید. (فرض کنید مشتق در نقطهٔ صفر برابر با صفر یا مشتق‌پذیرِ زیرگرادیانی در نظر گرفته می‌شود).

iii. با استفاده از بخش‌های قبل، ماتریس ژاکوبین $J_h(x)$ را برحسب ماتریس‌های A, C و بردار $Ax + b$ محاسبه کنید.

(ج) فرض کنید تابع f با تابع فعال‌ساز Softmax ترکیب شده باشد. برای یک بردار ورودی $z \in \mathbb{R}^m$، تابع Softmax به‌صورت زیر تعریف می‌شود:

$$s_i = \mathrm{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{m} e^{z_j}}, \quad \forall i \in \{1, \dots, m\}$$

اکنون فرض کنید $f(x) = \mathrm{Softmax}(Ax + b)$ و $g(z) = Cz$ باشند:

i. ضابطهٔ تابع ترکیبی $h(x) = g(f(x))$ را بنویسید.

ii. ماتریس ژاکوبین تابع Softmax یعنی $J_{\mathrm{Softmax}}(z) \in \mathbb{R}^{m\times m}$ را محاسبه کرده و نشان دهید که درایه‌های آن به‌صورت زیر هستند:

$$\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j)$$

که در آن δ_{ij} نماد دلتای کرونکر است. همچنین این ماتریس را برحسب بردار s و ماتریس قطری $\mathrm{diag}(s)$ به‌صورت ماتریسی فشرده بنویسید.

iii. با استفاده از قاعدهٔ زنجیره‌ای، ماتریس ژاکوبین نهایی $J_h(x)$ را برای این حالت محاسبه کنید.

۲. محاسبهٔ ژاکوبین و گرادیان تابع ضرر

فرض کنید $x \in \mathbb{R}^n$، $y \in \mathbb{R}^m$، $W \in \mathbb{R}^{m \times n}$، $b \in \mathbb{R}^m$ و تابع زیر تعریف شده باشد:

$$z = Wx + b$$

$$L(W, b, x) = \frac{۱}{۲}\|z - y\|^۲$$

(الف) ژاکوبین بردار z نسبت به بردار x را محاسبه کنید:

$$\frac{\partial z}{\partial x}$$

و اندازهٔ این ماتریس را مشخص کنید.

(ب) گرادیان تابع ضرر را نسبت به بردار بایاس محاسبه کنید:

$$\nabla_b L$$

(ج) گرادیان تابع ضرر را نسبت به ورودی محاسبه کنید:

$$\nabla_x L$$

(د) گرادیان تابع ضرر را نسبت به ماتریس وزن محاسبه کنید:

$$\nabla_W L$$

۳. پسانتشار خطا

فرض کنید یک شبکهٔ عصبی دو لایه به‌صورت زیر داریم:

$$z_۱ = W_۱ x^{(i)} + b_۱$$

$$a_۱ = \tanh(z_۱)$$

$$z_۲ = W_۲ a_۱ + b_۲$$

$$\hat{y}^{(i)} = z_۲$$

$$\mathcal{L}^{(i)} = \frac{۱}{۲}\left(\hat{y}^{(i)} - y^{(i)}\right)^۲$$

$$J = \frac{۱}{m}\sum_{i=۱}^{m} \mathcal{L}^{(i)}$$

در این‌جا، $x^{(i)}$ نشان‌دهندهٔ یک نمونهٔ ورودی منفرد با ابعاد $D_x \times ۱$ است. همچنین $y^{(i)}$ برچسب خروجی (هدف) متناظر با این نمونه بوده و یک اسکالر است. تعداد کل نمونه‌های داده برابر با m است. فرض می‌کنیم لایهٔ مخفی دارای $D_{a_۱}$ نود باشد؛ بنابراین $z_۱$ دارای ابعاد $D_{a_۱} \times ۱$ خواهد بود.

نکته: در بخش مربوط به پیاده‌سازی برداری (vectorized)، منظور این است که کل داده‌ها، یعنی هر m نمونه، به‌صورت همزمان به شبکه داده می‌شوند.

(الف) ابعاد ماتریس‌ها و بردارهای W_1، b_1، W_2 و b_2 را مشخص کنید. اگر شبکه را روی چندین نمونه به‌صورت برداری (vectorized) پیاده‌سازی کنیم، ابعاد وزن‌ها و بایاس‌ها چه تغییری می‌کنند؟ در حالت برداری، ابعاد X و Y چگونه خواهند بود؟

(ب) $\frac{\partial J}{\partial \hat{y}^{(i)}}$ را محاسبه کنید و آن را با $\delta_1^{(i)}$ نشان دهید.

(ج) $\frac{\partial \hat{y}^{(i)}}{\partial z_2}$ را محاسبه کنید و آن را با $\delta_2^{(i)}$ نشان دهید.

(د) $\frac{\partial z_2}{\partial a_1}$ را محاسبه کنید و آن را با $\delta_3^{(i)}$ نشان دهید.

(ه) $\frac{\partial a_1}{\partial z_1}$ را محاسبه کنید و آن را با $\delta_4^{(i)}$ نشان دهید.

(و) $\frac{\partial z_1}{\partial W_1}$ را محاسبه کنید و آن را با $\delta_5^{(i)}$ نشان دهید.

(ز) $\frac{\partial J}{\partial W_1}$ را محاسبه کنید. می‌توانید از نتایج قسمت‌های قبل استفاده کنید. راهنما: به ابعاد ماتریس‌ها دقت کنید.

۴. روش‌های بهینه‌سازی مرتبه اول

در این مسئله، تکامل الگوریتم‌های بهینه‌سازی مرتبه اول را بررسی می‌کنیم. هدف، تحلیل رفتار این الگوریتم‌ها برای کمینه‌سازی تابع محدب زیر است:

$$f(x,y) = 50x^2 + y^2$$

که در آن $x, y \in \mathbb{R}$ است. این تابع دارای انحنای بسیار زیاد در راستای محور x و انحنای کم در راستای محور y است، بنابراین یک مثال مناسب برای بررسی چالش‌های بهینه‌سازی محسوب می‌شود.

الگوریتم **Gradient Descent**

الگوریتم گرادیان کاهشی پارامترها را در جهت بیشترین کاهش تابع هدف به‌روزرسانی می‌کند:

$$x_{k+1} = x_k - \eta \nabla f(x_k)$$

که در آن $\eta > 0$ نرخ یادگیری است.

(الف) گرادیان تابع داده شده $\nabla f(x,y)$ را محاسبه کنید.

(ب) چرا استفاده از یک نرخ یادگیری ثابت η برای کمینه‌سازی این تابع، با توجه به تفاوت انحنای آن در راستای محورهای x و y، می‌تواند مشکل‌ساز باشد؟

(ج) با فرض $\eta = 0.1$ و نقطه اولیه $(x_0, y_0) = (1, 1)$، دو تکرار اول الگوریتم گرادیان کاهشی را محاسبه کنید. آیا این نتایج نشان‌دهنده مشکلات ذکرشده در بخش (ب) هستند؟

الگوریتم **Adaptive Learning Rates (AdaGrad)**

الگوریتم AdaGrad نرخ یادگیری هر پارامتر را بر اساس تاریخچه گرادیان‌های آن تنظیم می‌کند:

$$\eta_i^{(k)} = \frac{\eta}{\sqrt{\sum_{j=1}^{k} (g_i^{(j)})^2 + \epsilon}}$$

$$x_{i,k+1} = x_{i,k} - \eta_i^{(k)} g_i^{(k)}$$

که در آن $g_i^{(j)}$ مؤلفه iـام گرادیان در گام j است و $\epsilon > 0$ برای جلوگیری از تقسیم بر صفر است.

(د) چرا تنظیم نرخ یادگیری به‌صورت جداگانه توسط AdaGrad برای هر پارامتر در تابع $f(x, y)$ مفید است؟

(ه) با $\eta = 0.1$ و $\epsilon = 10^{-8}$، نرخ‌های یادگیری مؤثر $\eta_x^{(1)}$ و $\eta_y^{(1)}$ را بعد از اولین تکرار محاسبه کنید.

(و) نرخ یادگیری در طول زمان چه تغییری می‌کند و چه اثری دارد؟

الگوریتم **RMSProp**

الگوریتم RMSProp نسخه بهبودیافته AdaGrad است که به جای جمع کل گرادیان‌ها، از میانگین متحرک نمایی آن‌ها استفاده می‌کند:

$$s_{k+1} = \gamma s_k + (1 - \gamma) g_k^2$$

$$x_{k+1} = x_k - \frac{\eta}{\sqrt{s_{k+1} + \epsilon}} g_k$$

که در آن s_k میانگین متحرک نمایی مربع گرادیان‌ها و $\gamma \in [0, 1)$ ضریب کاهش است.

(ز) استفاده از میانگین متحرک نمایی مربع گرادیان‌ها در مقایسه با AdaGrad چه تفاوتی ایجاد می‌کند؟ یک مزیت واضح RMSProp نسبت به AdaGrad را بیان کنید.

(ح) نقش پارامتر γ در RMSProp را با نقش β در Momentum مقایسه کنید. هرکدام چه چیزی را کنترل می‌کنند؟

الگوریتم **Adam**

الگوریتم Adam ترکیبی از Momentum و AdaGrad است:

$$m_{k+1} = \beta_1 m_k + (1 - \beta_1) g_k$$

$$v_{k+1} = \beta_2 v_k + (1 - \beta_2) g_k^2$$

$$\hat{m}_{k+1} = \frac{m_{k+1}}{1 - \beta_1^k}, \quad \hat{v}_{k+1} = \frac{v_{k+1}}{1 - \beta_2^k}$$

$$x_{k+1} = x_k - \eta \frac{\hat{m}_{k+1}}{\sqrt{\hat{v}_{k+1}} + \epsilon}$$

(ط) بر اساس روابط بالا، مشخص کنید که کدام یک از متغیرهای m_k و v_k مربوط به Momentum و کدام مربوط به AdaGrad می‌باشند.

(ی) چرا در الگوریتم Adam نیاز به تصحیح بایاس برای m_k و v_k وجود دارد؟ اگر این تصحیح انجام نشود، در چند تکرار ابتدایی چه مشکلی رخ می‌دهد؟

(یالف) برای تابع $f(x, y) = 50x^2 + y^2$ نتیجه‌گیری کنید که چرا انتظار می‌رود Adam عملکرد بهتری نسبت به گرادیان کاهشی ساده داشته باشد؟

۵. منظم‌سازی (**Regularization**)

ماتریس طراحی X (که سطر iـام آن نقطهٔ نمونهٔ x_i^T است) و یک بردار nـتایی از برچسب‌ها $y = [y_1, \ldots, y_n]^T$ به شما داده شده است. برای سادگی، فرض کنید X سفید (whitened) شده است، به‌طوری که $X^T X = nI$ باشد. جملهٔ بایاس یا بعد اضافی اضافه نکنید؛ برای ورودی صفر، خروجی همیشه صفر است. فرض کنید x_{*i} ستون iـام ماتریس X را نشان می‌دهد.

(الف) نشان دهید که تابع هزینه برای کمترین مربعات با تنظیم‌گر ℓ_1 (Lasso)، یعنی:

$$J_1(w) \triangleq \|Xw - y\|^2 + \lambda \|w\|_1 \quad (\lambda > 0)$$

می‌تواند به‌صورت زیر بازنویسی شود:

$$J_1(w) = \|y\|^2 + \sum_{i=1}^{d} f(x_{*i}, w_i)$$

که در آن $f(\cdot,\cdot)$ یک تابع مناسب است که آرگومان اول آن یک بردار و آرگومان دوم آن یک اسکالر است.

(ب) با استفاده از پاسخ خود به بخش (۱)، شرایط لازم و کافی برای مولفهٔ iـام بهینه‌ساز w^* تابع $J_1(\cdot)$ را استخراج کنید به‌طوری که هر یک از سه خاصیت زیر را برآورده کند:

$$w_i^* > 0, \quad w_i^* = 0, \quad w_i^* < 0.$$

(ج) برای بهینه‌ساز $w^\#$ تابع هزینهٔ کمترین مربعات با تنظیم‌گر ℓ_2 (Ridge)، یعنی:

$$J_2(w) \triangleq \|Xw - y\|^2 + \lambda \|w\|_2^2 \quad (\lambda > 0)$$

یک شرط لازم و کافی برای $w_i^\# = 0$ استخراج کنید، که در آن $w_i^\#$ مولفهٔ iـام بردار $w^\#$ است.

(د) یک بردار را تُنُک (Sparse) می‌نامیم اگر بیشتر مولفه‌های آن صفر باشند. بر اساس پاسخ‌های خود به بخش‌های (۲) و (۳)، کدام یک از بردارهای w^* و $w^\#$ به احتمال زیاد تُنُک‌تر است؟ چرا؟

۶. هموردایی نسبت به انتقال (**Translation Equivariance**) و عملگرهای کانولوشن

فرض کنید $I \in \mathbb{R}^{H \times W}$ یک تصویر و عملگر انتقال (Translation Operator) را به‌صورت زیر تعریف می‌کنیم:

$$(T_{a,b} I)(i,j) = I(i-a, j-b)$$

همچنین یک عملگر L را هموردا نسبت به انتقال (Translation Equivariant) گویند اگر برای هر انتقال (a,b) داشته باشیم:

$$L(T_{a,b} I) = T_{a,b}(L(I))$$

همچنین کانولوشن با هسته $K \in \mathbb{R}^{k \times k}$ به‌صورت زیر تعریف می‌شود:

$$(K * I)(i,j) = \sum_{m,n} K(m,n) I(i-m, j-n)$$

(الف) نشان دهید کانولوشن نسبت به انتقال هموردا است.

(ب) ثابت کنید هر عملگر خطی که نسبت به انتقال هموردا است، یک کانولوشن است. توضیح دهید این نتیجه چه تأثیری بر انتخاب شبکه‌های کانولوشنی و طراحی آن‌ها دارد.

(ج) فرض کنید تصاویر قبل ورود به شبکه به بردار $x \in \mathbb{R}^{HW}$ تبدیل می‌شوند. ابتدا نشان دهید که ماتریس W متناظر با تبدیل خطی که نسبت به انتقال هموردا است، یک ماتریس توپلیتز (Toeplitz) است. سپس توضیح دهید چرا یک لایهٔ خطی

موجود در MLP به‌طور کلی نمی‌تواند نسبت به انتقال هموردا باشد.

(د) فرض کنید از یک شبکهٔ عصبی دو لایه برای تشخیص یک الگو در یک تصویر استفاده می‌کنیم. حداقل تعداد پارامتر قابل یادگیری مورد نیاز در لایهٔ اول برای تشخیص یک الگوی $k \times k$ را به‌ازای لایهٔ اول کانولوشنی و خطی کاملاً متصل محاسبه و مقایسه کنید.

تعریف: ماتریس Toeplitz، ماتریسی است که درایه‌های هر قطر موازی با قطر اصلی، با هم برابر باشند. به عبارت دیگر مقدار درایه‌ها تابعی از اختلاف شماره سطر و ستون آن است. برای مثال ماتریس T که به‌صورت زیر تعریف شده، یک ماتریس Toeplitz است:

$$T = \begin{bmatrix} T_۰ & T_{-۱} & T_{-۲} & T_{-۳} \\ T_۱ & T_۰ & T_{-۱} & T_{-۲} \\ T_۲ & T_۱ & T_۰ & T_{-۱} \\ T_۳ & T_۲ & T_۱ & T_۰ \end{bmatrix}$$